

Java & OOP Fundamentals

Interview-Prep Notes and Practice Workbook — WSO2 Engineering Internship

K. R. T. Uthsara

BSc (Hons) Computer Science, University of Ruhuna

Contents

0. Setting up your practice environment
1. The four OOP pillars
2. Inheritance vs composition
3. Interfaces vs abstract classes
4. How HashMap works internally
5. Exceptions
6. Threads and synchronization
7. Three-week study plan
8. Self-test interview questions (with answers)
9. Final readiness checklist

0. Setting Up Your Practice Environment

Every section in these notes ends with practice tasks. Reading is not enough for interviews — you need to **run the code, break it, and fix it**. Set this up once before you start.

What to install

- **JDK 21 (LTS)** — Eclipse Temurin from `adoptium.net` is free and works everywhere. Some examples use pattern matching (`o instanceof Student s`), which needs Java 16 or newer.
- **IntelliJ IDEA Community Edition** (free) — best for Java. Ctrl+Click on any class (e.g. `HashMap`) opens its real JDK source code, which you will need in Section 4. VS Code with the "Extension Pack for Java" also works.
- **Git + a GitHub repo** called something like `java-interview-practice` . Commit each practice task. It becomes evidence of your preparation, and WSO2 values visible work on GitHub.

Check the install

```
java -version      # should print 21.x
javac -version
```

Fastest way to run a single file

From Java 11 onward you can run a one-file program directly, without compiling first:

```
java RaceDemo.java
```

For multi-file tasks, put them in one folder and compile everything:

```
javac *.java
java Main
```

Suggested folder structure

```
java-interview-practice/
├── 01-oop/
├── 02-inheritance-composition/
├── 03-interfaces/
├── 04-hashmap/
├── 05-exceptions/
├── 06-threads/
└── NOTES.md      <- write each topic's explanation in your own words here
```

සිංහලෙන් මේ notes කියවලා විතරක් මදි. හැම code එකක්ම ඔයාම type කරලා run කරන්න, වැරදි කරලා බලන්න, ඊට පස්සේ fix කරන්න. GitHub repo එකකට commit කරන එකෙන් ඔයාගේ වැඩ පෙන්නන්නත් පුළුවන්.

1. The Four OOP Pillars

Interviewers often open with this. Have a one-sentence answer and a tiny example ready for each.

Pillar	One-sentence definition	Java mechanism
Encapsulation	Hide an object's data; allow access only through its methods.	<code>private</code> fields + public methods
Abstraction	Expose <i>what</i> something does, hide <i>how</i> it does it.	Interfaces, abstract classes
Inheritance	A class reuses and extends another class's behavior.	<code>extends</code>
Polymorphism	One reference type, different behavior at runtime depending on the actual object.	Method overriding (runtime), overloading (compile time)

Pillars.java

```
class BankAccount {
    private double balance;                // Encapsulation: hidden

    public void deposit(double amount) {
        if (amount <= 0) throw new IllegalArgumentException("Amount must be
positive");
        balance += amount;                // only changed through a checked method
    }
    public double getBalance() { return balance; }
}

abstract class Shape { abstract double area(); }           // Abstraction
class Circle extends Shape {                             // Inheritance
    double r; Circle(double r) { this.r = r; }
    double area() { return Math.PI * r * r; }              // Polymorphism (override)
}
class Square extends Shape {
    double s; Square(double s) { this.s = s; }
    double area() { return s * s; }
}

// Polymorphism in action: same call, different behavior
Shape[] shapes = { new Circle(1), new Square(2) };
for (Shape sh : shapes) System.out.println(sh.area());
```

සිංහලෙන් **Encapsulation** = දත්ත `private` කරලා methods හරහා විතරක් වෙනස් කරන්න දීම.
Polymorphism = එකම method call එක (`sh.area()`), ඇත්ත object එක අනුව වෙනස් විදිහට වැඩ කිරීම.

Overloading vs overriding: **Overloading** = same method name, different parameters, same class — decided at *compile* time. **Overriding** = child class replaces a parent method with the same signature — decided at *run* time. This distinction is a very common follow-up question.

Practice 1 Create `BankAccount` and try to set `balance` directly from `main`. Read the compiler error. Then add a `withdraw()` method that refuses to go below zero.

Practice 2 Add a `Triangle` class to the `Shape` example without changing the loop. Explain out loud why the loop didn't need to change — that *is* polymorphism.

2. Inheritance vs Composition

Inheritance = "is-a". A Dog *is an* Animal.

Composition = "has-a". A Car *has an* Engine.

Composition.java

```
// Inheritance: Dog IS an Animal
class Animal { void eat() { System.out.println("eating"); } }
class Dog extends Animal { void bark() { System.out.println("woof"); } }

// Composition: Car HAS an Engine
interface Engine { void start(); }
class PetrolEngine implements Engine { public void start()
{ System.out.println("petrol engine on"); } }
class ElectricEngine implements Engine { public void start()
{ System.out.println("electric motor on"); } }

class Car {
    private final Engine engine;
    Car(Engine engine) { this.engine = engine; }    // engine is passed in
    void drive() { engine.start(); }
}

new Car(new PetrolEngine()).drive();
new Car(new ElectricEngine()).drive();    // swapped without touching Car
```

Why "favor composition over inheritance"?

- **Tight coupling.** A child class depends on the parent's internal behavior. Change the parent and children can break — this is called the **fragile base class** problem.
- **Only one parent.** Java allows a class to `extend` only one class, so inheritance uses up your only slot.
- **Flexibility.** With composition you can swap the part (petrol → electric engine) at runtime.

Classic bad example: class Stack extends ArrayList. A stack should only allow push/pop, but by inheriting from ArrayList it also exposes add(index, x) and remove(index), letting anyone insert into the middle. Java's own old java.util.Stack extends Vector has exactly this flaw. Mention this in an interview — it shows you understand the problem, not just the slogan.

සිංහලෙන් Inheritance = "මේක අරකේ වර්ගයක්" (is-a). Composition = "මේක ඇතුළේ අරක තියෙනවා" (has-a). Parent class එක වෙනස් වුණොත් inheritance වලදී child classes කැඩෙන්න පුළුවන්. Composition වලදී කොටස (engine එක) ලේසියෙන් මාරු කරන්න පුළුවන් — ඒ නිසා ඒක වඩා නම්‍යශීලීයි.

Interview answer: "I use inheritance only for a true is-a relationship where the child can fully replace the parent. Otherwise I use composition, because it gives looser coupling and lets me swap implementations."

Practice 3 Write class `MyStack` extends `ArrayList<Integer>` with `push/pop`. Show that `add(0, 99)` breaks the stack's rules. Then rewrite it using composition (a *private* `ArrayList` field) so only `push`, `pop` and `peek` are possible.

3. Interfaces vs Abstract Classes

Interfaces.java

```
interface Payable {
    double calculatePay();           // abstract by default
    default String currency() { return "LKR"; } // Java 8+: default method
    static Payable zero() { return () -> 0; } // Java 8+: static method
}

abstract class Employee {
    protected final String name;           // can hold state
    Employee(String name) { this.name = name; } // can have a constructor
    abstract double calculatePay();
    String describe() { return name + " earns " + calculatePay(); } // shared code
}

class Manager extends Employee implements Payable, Comparable<Manager> {
    Manager(String n) { super(n); }
    public double calculatePay() { return 150_000; }
    public int compareTo(Manager o) { return name.compareTo(o.name); }
}
```

	Interface	Abstract class
How many can one class use?	Many (implements A, B, C)	Only one (extends)
Instance fields (state)?	No — only public static final constants	Yes
Constructor?	No	Yes
Method bodies?	Only default, static, private (Java 8/9+)	Yes, any
Meaning	A capability — "can be paid", "can be compared"	A base type — "is an employee"

සිංහලෙන් Interface = "මේ class එකට මේ දේ කරන්න පුළුවන්" කියන පොරොන්දුවක් (contract). එක class එකකට interfaces ගොඩක් implement කරන්න පුළුවන්. Abstract class = පොදු state (fields) එකයි code එකයි බෙදාගන්න base එකක්, හැබැයි එකක් විතරයි extend කරන්න පුළුවන්.

Follow-up — why did Java 8 add default methods? So new methods (like `forEach` and `stream`) could be added to existing interfaces such as `List` and `Collection` without breaking every class in the world that already implemented them.

Follow-up — the diamond problem: If a class implements two interfaces that both have a default method with the same signature, the code won't compile until the class overrides that method itself. It can call a specific one with `A.super.method()`.

Practice 4 Create two interfaces `Flyer` and `Swimmer`, each with default `void move()`. Make `Duck` implements `Flyer`, `Swimmer`. Read the compiler error, then fix it by overriding `move()` and calling `Flyer.super.move()`.

Practice 5 Sort a `List<Manager>` with `Collections.sort()`. Then sort the same list by pay using a `Comparator` lambda. Explain the difference between `Comparable` (natural order, inside the class) and `Comparator` (external order, any number of them).

4. How HashMap Works Internally

This is one of the most frequently asked Java questions. Know every step, in order.

Structure

An **array of buckets**. Each bucket holds a linked list of entries (key, value, hash, next) — or a red-black tree if it grows large.

What happens on `put(key, value)`

1. Call `key.hashCode()` → an `int h`.
2. **Spread the bits:** `hash = h ^ (h >>> 16)`. This mixes the high bits into the low bits so that they affect the bucket choice.
3. **Find the bucket:** `index = hash & (n - 1)`, where `n` is the array length. `n` is always a power of 2, so this is a fast replacement for `hash % n`.
4. Bucket empty → store the entry there.
5. Bucket occupied (a **collision**) → walk the bucket comparing keys with `equals()`. Same key → replace the value. New key → append.
6. **Treeify:** if one bucket has more than **8** entries *and* the table has at least **64** buckets, the list becomes a red-black tree. (If the table is smaller than 64, it resizes instead.)
7. **Resize:** default capacity **16**, load factor **0.75**. When size exceeds `capacity × 0.75` ($16 × 0.75 = 12$), the array doubles to 32 and entries are redistributed.

`get(key)`

Same steps 1–3 to find the bucket, then `equals()` to find the right entry inside it.

Operation	Average	Worst case (before Java 8)	Worst case (Java 8+)
<code>get / put</code>	$O(1)$	$O(n)$ — long list	$O(\log n)$ — tree

සිංහලෙන් HashMap එක ඇතුළේ තියෙන්නේ buckets array එකක්. Key එකේ `hashCode()` එකෙන් bucket index එක නොයනවා. එකම bucket එකට keys දෙකක් වැටුණොත් (collision), ඒවා list එකක තියාගෙන `equals()` වලින් හරි key එක නොයනවා. List එක 8ට වඩා දිග වුණොත් tree එකක් බවට හැරෙනවා. 75% පිරුණාම array එක දෙගුණ වෙනවා.

The equals / hashCode contract — the trap question

HashTrap.java

```
import java.util.*;

class Student {
    String id;
    Student(String id) { this.id = id; }

    @Override public boolean equals(Object o) {
        return o instanceof Student s && id.equals(s.id);
    }
    // hashCode() NOT overridden <-- the bug
}

public class HashTrap {
    public static void main(String[] args) {
        Map<Student, Integer> marks = new HashMap<>();
        marks.put(new Student("SC001"), 85);
        System.out.println(marks.get(new Student("SC001"))); // prints null !
    }
}
```

The two `Student` objects are `equals`, but without an overridden `hashCode()` each gets a different default hash code. They land in **different buckets**, so `get` never even looks in the right place.

The rule: if you override `equals()`, you must override `hashCode()`. Equal objects **must** have equal hash codes. (Unequal objects *may* share a hash code — that's just a collision.)

```
@Override public int hashCode() { return Objects.hash(id); }
```

සිංහලෙන් `equals()` override කළොත් `hashCode()` එකත් අනිවාර්යයෙන්ම override කරන්න ඕන. නැත්නම් "සමාන" objects දෙකක් වෙනස් buckets වලට ගිහින්න `get()` එකෙන් `null` එනවා.

Two more traps: (1) Mutable keys. If you change a field used in `hashCode()` after putting the object in a map, it is now in the wrong bucket and effectively lost. Use immutable keys (`String`, `Integer`, records). **(2) Collisions are real:** `"Aa".hashCode()` and `"BB".hashCode()` are both 2112.

Quick facts

- One `null` key is allowed (it goes to bucket 0). `Hashtable` and `ConcurrentHashMap` allow none.
- Iteration order is **not** guaranteed. Use `LinkedHashMap` for insertion order, `TreeMap` for sorted order.
- **Not thread-safe.** Use `ConcurrentHashMap` when several threads share the map.

Practice 6 Run `HashTrap.java` and see `null`. Add `hashCode()` and confirm it prints 85. Then print `"Aa".hashCode()` and `"BB".hashCode()` to see a real collision.

Practice 7 In IntelliJ, Ctrl+Click on `HashMap`, find the `hash()` and `putVal()` methods, and match each line to the steps above. Also find the constants `TREEIFY_THRESHOLD` and `DEFAULT_LOAD_FACTOR`.

Practice 8 **Stretch goal (strong interview story):** write your own `MyHashMap<K, V>` with an array of linked-list buckets, `put`, `get`, and `resize` at 0.75 load. About 60 lines. If you can build it, you can explain it.

5. Exceptions

The hierarchy

```

Throwable
├── Error                                JVM problems (OutOfMemoryError, StackOverflowError) -
don't catch
├── Exception
│   ├── IOException, SQLException, ...    <- CHECKED
│   └── RuntimeException
│       ├── NullPointerException
│       ├── IllegalArgumentException      <- UNCHECKED
│       └── ArrayIndexOutOfBoundsException, ...

```

	Checked	Unchecked
Compiler forces handling?	Yes — catch it or declare <code>throws</code>	No
Typical cause	Something outside your control: file missing, network down	A bug in your code: null access, bad index
Superclass	<code>Exception</code> (but not <code>RuntimeException</code>)	<code>RuntimeException</code>

සිංහලෙන් **Checked exceptions** = compiler එක බල කරනවා handle කරන්න (file නැති වීම වගේ පිටින් එන ප්‍රශ්න). **Unchecked** = programmer ගේ වැරදි (null එකක් පාවිච්චි කිරීම වගේ) — compiler එක බල කරන්නේ නෑ. **Error** = JVM එකේ ප්‍රශ්න, අල්ලගන්න යන්න එපා.

try-with-resources

Any object that implements `AutoCloseable` is closed automatically — even if an exception is thrown.

ReadFile.java

```

try (BufferedReader br = new BufferedReader(new FileReader("data.txt"))) {
    System.out.println(br.readLine());
} catch (FileNotFoundException e) {           // more specific catch FIRST
    System.err.println("File not found: " + e.getMessage());
} catch (IOException e) {
    System.err.println("Could not read file: " + e.getMessage());
}
// br.close() has already been called here, automatically

```

Custom exceptions

CustomException.java

```
class InsufficientBalanceException extends Exception {           // checked
    InsufficientBalanceException(String msg) { super(msg); }
}

void withdraw(double amount) throws InsufficientBalanceException {
    if (amount > balance)
        throw new InsufficientBalanceException("Balance " + balance + " < " + amount);
    balance -= amount;
}
```

Never do this: `catch (Exception e) { }` — an empty catch "swallows" the error and hides bugs. Always log it, handle it, or rethrow it. Also avoid catching `Exception` when you mean a specific type.

Trick questions about `finally`

FinallyTest.java

```
static int test() {
    try {
        return 1;
    } finally {
        System.out.println("finally runs");    // YES, this prints before returning
    }
}
```

- **Does `finally` run after `return`?** Yes. Only `System.exit()` or a JVM crash stops it.
- **What if `finally` also has a `return`?** It overrides the `try`'s return value and silently discards any exception. Never write this — but know it.
- **`throw` vs `throws`?** `throw` actually throws an exception object. `throws` in a method signature declares which checked exceptions it may throw.

Practice 9 Add `InsufficientBalanceException` to your `BankAccount` from Practice 1. Write a `main` that catches it and prints a friendly message.

Practice 10 Run `FinallyTest.java`. Then put `return 2;` inside the `finally` block and predict the result before running it.

Practice 11 Write a class that implements `AutoCloseable` and prints "closed" in `close()`. Use it in `try-with-resources` and throw an exception inside the `try`. Confirm "closed" still prints.

6. Threads and Synchronization

Past WSO2 intern candidates report being asked about Java threads and concurrency specifically, so give this section the most time. WSO2's API gateways handle thousands of concurrent requests — this is the everyday reality of their codebase.

Creating threads

```
Thread t = new Thread(() ->
    System.out.println("running in " + Thread.currentThread().getName()));
t.start();      // starts a NEW thread
// t.run();     // WRONG: just calls the method on the CURRENT thread
t.join();       // wait for t to finish
```

Why prefer Runnable / lambda over extends Thread? Extending Thread uses up your one parent class, and mixes *the task* with *the mechanism that runs it*. A Runnable can be run by a thread, a thread pool, or anything else.

Race condition — run this yourself

RaceDemo.java

```
public class RaceDemo {
    static int count = 0;

    public static void main(String[] args) throws InterruptedException {
        Runnable task = () -> {
            for (int i = 0; i < 100_000; i++) count++;
        };
        Thread t1 = new Thread(task);
        Thread t2 = new Thread(task);
        t1.start(); t2.start();
        t1.join(); t2.join();
        System.out.println("Expected 200000, got " + count);
    }
}
```

Run it several times — you'll get a different number below 200000 almost every time. `count++` looks like one step but is really three: **read** the value, **add 1**, **write** it back. Two threads can both read 500, both compute 501, and both write 501. One increment is lost.

සිංහලෙන් `count++` කියන්නේ එක පියවරක් නෙවෙයි — කියවනවා, එකතු කරනවා, ලියනවා. Threads දෙකක් එකම වෙලාවේ පරණ අගයම කියවලා ලිව්වොත් එක increment එකක් නැති වෙනවා. මේකට කියන්නේ **race condition**.

Three fixes — and when to use each

```
// 1. synchronized: only one thread at a time may enter
static synchronized void increment() { count++; }

// 2. AtomicInteger: lock-free, fastest for a simple counter
static AtomicInteger count = new AtomicInteger();
count.incrementAndGet();

// 3. ReentrantLock: more control (tryLock, timeouts, fairness)
static final Lock lock = new ReentrantLock();
lock.lock();
try { count++; } finally { lock.unlock(); }    // ALWAYS unlock in finally
```

Tool	Use when
synchronized	Simple, general mutual exclusion. Released automatically, even on exception.
AtomicInteger etc.	A single variable updated by many threads (counters, flags). Fastest.
ReentrantLock	You need tryLock(), a timeout, or fair ordering. You must unlock manually.

volatile — the trap question

volatile guarantees **visibility** (every thread sees the most recent write) but **not atomicity**. volatile int count; count++; is *still* a race condition. Use volatile for simple flags:

```
private volatile boolean running = true;

void runLoop() { while (running) { /* work */ } }
void stop()    { running = false; }    // other thread sees this immediately
```

සිංහලෙන් volatile එකෙන් හැම thread එකකටම අලුත්ම අගය පේනවා (**visibility**), හැබැයි count++ වගේ පියවර කිහිපයක operation එකක් ආරක්ෂා කරන්නේ නෑ (**atomicity** නෑ).

Deadlock — run this too

DeadlockDemo.java

```
public class DeadlockDemo {
    static final Object lock1 = new Object();
    static final Object lock2 = new Object();

    public static void main(String[] args) {
        new Thread(() -> {
            synchronized (lock1) {
                pause(100);
                synchronized (lock2) { System.out.println("A done"); }
            }
        }).start();

        new Thread(() -> {
            synchronized (lock2) {           // opposite order!
                pause(100);
                synchronized (lock1) { System.out.println("B done"); }
            }
        }).start();
    }

    static void pause(long ms) {
        try { Thread.sleep(ms); } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}
```

The program hangs forever: Thread A holds `lock1` and waits for `lock2`; Thread B holds `lock2` and waits for `lock1`.

The four conditions for deadlock (all four must hold): mutual exclusion, hold-and-wait, no preemption, circular wait. **Standard fix:** break circular wait by always acquiring locks in the **same global order**.

Alternatively use `tryLock()` with a timeout.

සිංහලෙන් **Deadlock** = threads දෙකක් එකිනෙකා අල්ලගෙන ඉන්න lock එක එනකම් සදාකාලිකව බලාගෙන ඉන්න එක. විසඳුම: program එකේ හැම තැනම locks ගන්නේ එකම පිළිවෙලට.

Thread pools — how real code does it

PoolDemo.java

```
ExecutorService pool = Executors.newFixedThreadPool(4);
for (int i = 0; i < 10; i++) {
    int id = i;                                     // must be effectively final for the
    lambda
    pool.submit(() -> System.out.println("task " + id + " on " +
    Thread.currentThread().getName()));
}
pool.shutdown();                                   // no new tasks; finish queued ones
pool.awaitTermination(5, TimeUnit.SECONDS);
```

Creating a thread is expensive. A pool keeps a fixed set of threads alive and feeds them tasks from a queue. Use `submit()` with a `Callable` to get a `Future` result back.

Practice 12 Run RaceDemo five times and record the results. Fix it all three ways and confirm you get 200000 every time.

Practice 13 Change count to `volatile int` without any other fix. Run it. Explain why it is still wrong.

Practice 14 Run DeadlockDemo. While it hangs, take a thread dump (IntelliJ: camera icon in the Run window, or `jstack <pid>` from the terminal) — the JVM reports "Found one Java-level deadlock". Then fix it by making both threads take `lock1` first.

Practice 15 Rewrite RaceDemo using a thread pool of 4 threads submitting 4 tasks, with `AtomicInteger`.

7. Three-Week Study Plan

Week	Topics	Practice tasks
1	OOP pillars · inheritance vs composition · interfaces vs abstract classes	1, 2, 3, 4, 5
2	HashMap internals · equals/hashCode · reading JDK source	6, 7, and 8 (stretch)
3	Exceptions · threads · synchronization · deadlock · thread pools	9 – 15

Daily routine (about 1.5–2 hours)

1. **20 min** — read one section of these notes.
2. **40 min** — do its practice tasks. Type the code; don't copy-paste.
3. **15 min** — close everything and explain the topic **out loud** as if to an interviewer. Record yourself on your phone if you can.
4. **10 min** — write the explanation in your own words in `NOTES.md` and commit.
5. **20–30 min** — one LeetCode easy/medium problem in Java (arrays, strings, hashing first).

සිංහලෙන් "Out loud" test එක තමයි ඇත්ත පරීක්ෂණය. Code run කරන එකෙන් පේන්නේ ඔයාට පාවිච්චි කරන්න පුළුවන් කියන එක. Notes නැතුව පැහැදිලි කරන්න පුළුවන් නම් තමයි ඔයාට තේරිලා තියෙන්නේ — interview එකේ බලන්නේ ඒක. අමාරු වුණු තැන තමයි ආයෙත් කියවන්න ඕන කොටස.

8. Self-Test Interview Questions

Cover the answer, say yours out loud, then check.

Q1. What is the difference between overloading and overriding?

Overloading: same name, different parameter lists, resolved at compile time. Overriding: subclass redefines a parent method with the same signature, resolved at runtime (dynamic dispatch).

Q2. Why favor composition over inheritance?

Inheritance couples the child to the parent's internals (fragile base class), and Java allows only one parent. Composition is looser, lets you swap implementations, and exposes only what you choose.

Q3. Interface or abstract class — how do you choose?

Interface for a capability many unrelated classes can have, especially when a class needs several. Abstract class when related classes share state and common code.

Q4. Explain how `HashMap.put` works.

`hashCode` → spread bits ($h \oplus h \gg 16$) → `index = hash & (n-1)` → empty bucket: insert; otherwise compare with equals and replace or append. Treeify above 8 per bucket (`table ≥ 64`). Resize $\times 2$ when `size > capacity \times 0.75`.

Q5. What happens if you override `equals` but not `hashCode`?

Equal objects get different hash codes, land in different buckets, and lookups fail — `get` returns null for a key that is logically present.

Q6. Is `HashMap` thread-safe? What do you use instead?

No. Concurrent modification can corrupt it. Use `ConcurrentHashMap` (or `Collections.synchronizedMap` for simple cases).

Q7. Checked vs unchecked exceptions?

Checked extend `Exception` (not `RuntimeException`) and must be caught or declared — for recoverable external problems. Unchecked extend `RuntimeException` — usually programming bugs, not forced.

Q8. Does `finally` run if `try` returns?

Yes, always, unless the JVM exits (`System.exit`) or crashes. A return inside `finally` overrides the `try`'s return — bad practice.

Q9. `start()` vs `run()`?

`start()` creates a new thread which then calls `run()`. Calling `run()` directly just executes it on the current thread — no concurrency.

Q10. Why is `count++` not thread-safe? How do you fix it?

It is read-modify-write, three steps, so updates can be lost. Fix with `synchronized`, `AtomicInteger`, or a `Lock`.

Q11. What does `volatile` guarantee?

Visibility and ordering of reads/writes across threads. Not atomicity — `volatile count++` is still a race.

Q12. What is a deadlock and how do you prevent it?

Threads each waiting forever for a lock another holds. Needs mutual exclusion, hold-and-wait, no preemption, and circular wait. Prevent by acquiring locks in a consistent global order, or using `tryLock` with a timeout.

Q13. Why use a thread pool?

Creating threads is expensive. A pool reuses a fixed set of threads, limits concurrency, and queues tasks — essential for servers handling many requests.

9. Final Readiness Checklist

Tick an item only when you can explain it **without notes**.

- ☐ Define the four OOP pillars with an example of each
- ☐ Explain overloading vs overriding
- ☐ Explain is-a vs has-a and the fragile base class problem
- ☐ Give the Stack-extends-ArrayList counterexample
- ☐ Compare interfaces and abstract classes (5 differences)
- ☐ Explain default methods and why Java 8 added them
- ☐ Walk through HashMap.put step by step, including hash spreading and index calculation
- ☐ State the load factor, default capacity and treeify threshold
- ☐ Explain the equals/hashCode contract and demonstrate the bug
- ☐ Draw the exception hierarchy from memory
- ☐ Explain checked vs unchecked with an example of each
- ☐ Write try-with-resources from memory
- ☐ Answer the finally-with-return trick questions
- ☐ Explain why count++ is not atomic
- ☐ Fix a race condition three ways and say when to use each
- ☐ Explain what volatile does and does not guarantee
- ☐ State the four deadlock conditions and how to prevent deadlock
- ☐ Take a thread dump of a deadlocked program
- ☐ Use ExecutorService and explain why pools exist
- ☐ All 15 practice tasks committed to GitHub

Reliable resources

- **dev.java/learn** — Oracle's official, up-to-date Java tutorials.
- **The JDK source itself** — `HashMap.java`, `ConcurrentHashMap.java`, `ReentrantLock.java` (Ctrl+Click in IntelliJ).
- **Effective Java** (Joshua Bloch, 3rd ed.) — Items 10–11 (equals/hashCode), 18 (composition), 20 (interfaces), 69–77 (exceptions), 78–84 (concurrency).
- **Java Concurrency in Practice** (Brian Goetz) — the standard book on threads, if you want to go deeper.